LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – Inheritance



Farrel Labib Alhaidar

103132400009

S1SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Konsep Dasar Inheritance (Pewarisan)

Inheritance adalah mekanisme di mana sebuah kelas baru (disebut subclass atau derived class) dibuat berdasarkan kelas yang sudah ada (disebut superclass atau base class). Subclass akan mewarisi seluruh atribut (state) dan metode (behaviour) dari superclass-nya.

Tujuan Utama: Menghindari duplikasi kode (reusability). Alih-alih menulis ulang kode yang sama di setiap kelas, kita cukup mendefinisikannya sekali di superclass, dan subclass otomatis.

1. Keyword super (Mengakses Superclass)

   Super adalah kata kunci khusus yang digunakan di dalam subclass untuk merujuk secara eksplisit ke anggota (atribut, metode, atau konstruktor) milik superclass-nya.

   Tiga Fungsi Utama super:

   - Memanggil Konstruktor Superclass: super(parameter) digunakan di baris pertama konstruktor subclass untuk menginisialisasi atribut yang ada di superclass.
   - Mengakses Atribut Superclass: super.namaAtribut digunakan jika subclass memiliki atribut dengan nama yang sama (shadowing), sehingga kita perlu merujuk ke versi superclass.
   - Memanggil Metode Superclass: super.namaMetode() digunakan untuk memanggil metode asli dari superclass yang mungkin telah di-override oleh subclass.

2. Overloading Method (Metode Berlebih)

   Kemampuan untuk membuat beberapa metode dengan nama yang sama di dalam satu kelas, asalkan daftar parameternya berbeda (jumlah, tipe data, atau urutan parameter).

   Nama metode sama, tapi signature (parameter) harus unik. Tipe kembalian (return type) tidak bisa dijadikan pembeda overloading.

   Tujuan: Memberikan fleksibilitas pada metode yang fungsinya mirip namun menerima input berbeda.

3. Overriding Method (Metode Menimpa)

   Kemampuan sebuah subclass untuk mendefinisikan ulang metode yang sudah ada di superclass dengan nama, parameter, dan tipe kembalian yang sama persis.

   Terjadi antar kelas (superclass dan subclass), bukan dalam satu kelas. Saat metode dipanggil dari objek subclass, maka metode versi subclass-lah yang akan dieksekusi (mengalahkan versi superclass).

   Tujuan: Subclass ingin memberikan perilaku yang lebih spesifik atau berbeda dari perilaku umum yang didefinisikan superclass.

B. Guided

1. Guided1

A.java

```java
package guided.guided1;


public class A {

    int x;

    int y;


    void tampilXY(){

        System.out.println("Nilai x = " + x + ", y = " + y);

    }

}
```

B.java

```java
package guided.guided1;

public class B extends A{
    int z;
    int hasil;

    void tampilXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

Demo.java

```java
package guided.guided1;
```

```java
public class Demo {
    public static void main(String[] args) {

        A superClass = new A();
        B subClass = new B();

        superClass.x = 3;
        superClass.y = 4;

        System.out.println("superclass: ");
        superClass.tampilXY();

        subClass.x = 1;
        subClass.y = 2;
        subClass.z = 5;

        System.out.println("subclass: ");
        subClass.tampilXYZ();
    }
}
```

Output :

```
superclass:
Nilai x = 3, y = 4
subclass:
Jumlah (x+y+z) = 8
```

2. Guided2

Employes.java

```java
package guided.guided2;

public class Employes {
    private String nama;
    String department;

    public Employes(String nama) {
```

```
        this.nama = nama;
    }
    public void TampilNama(){
        System.out.println("Nama: " + nama);
    }
}
```

Manager.java

```
package guided.guided2;

public class Manager extends Employes {
    private String alamat;

    public Manager(String nama, String alamat) {
        super(nama);
        this.alamat = alamat;
    }
    public void tampilInformasi(){
        super.TampilNama();
        System.out.println("Alamat: " + alamat);
        System.out.println("Department: " + super.department);
    }
}
```

Demo.java

```
package guided.guided2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.department = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
Nama: Adi
Alamat: Sukabirus
Department: SDM
```

3. Guided3

Lagu.java

```
package guided.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }
    void IsiParam(String param1,String param2) {
        judul = param1;
        pencipta = param2;
    }
    void tampilkan(){
        System.out.println("Judul lagu: " + judul + "Pencipta lagu: " +
pencipta);
    }
}
```

Main.java

```
package guided.guided3;

public class Main {
    public static void main(String[] args) {
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();
```

```
        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
Judul lagu: Lagu 1Pencipta lagu: Tidak dikenal
Judul lagu: Lagu 2Pencipta lagu: aku
```

4. Guided4

A.java

```
package guided.guided4;

public class A {
    public void tampilkanKeLayar() {
    System.out.println("Method milik class A dipanggil...");
    }
}
```

B.java

```
package guided.guided4;

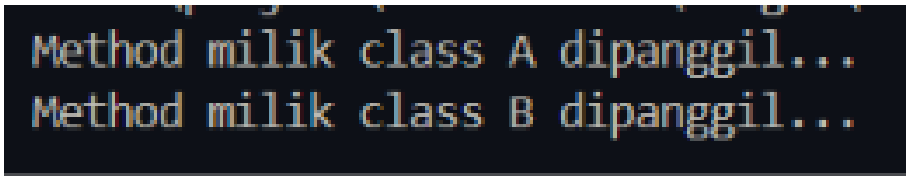
public class B extends A {
    public void tampilkanKeLayar() {
    super.tampilkanKeLayar();
    System.out.println("Method milik class B dipanggil...");
    }
}
```

Main.java

```
package guided.guided4;

public class Main {
public static void main(String[] args) {
    B subOb = new B();
    subOb.tampilkanKeLayar();
    }
}
```

Output:



C. Unguided

1. Soal Unguided

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).

Memiliki constructor yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.

Memiliki method untuk menampilkan informasi.

Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (overloading):

Tanpa parameter: biaya = jumlah data latih × 2.

Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

Mewarisi kelas induk dan tambahkan atribut jumlah fitur.

Constructor menerima nama, jumlah data latih, dan jumlah fitur; gunakan constructor kelas induk.

Override method: tampilkan informasi dari kelas induk.

Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).

Tambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, tetapi menerima dua parameter bilangan bulat (jumlah epoch dan ukuran batch): biaya = jumlah data latih × epoch ÷ ukuran batch.
Sehingga objek kelas ini memiliki tiga versi method penghitung biaya.
3. Kelas turunan kedua (model klasifikasi)
Mewarisi kelas induk dan menambahkan atribut jumlah kelas.
Constructor menerima nama, jumlah data latih, dan jumlah kelas; gunakan constructor kelas induk.
Override method: tampilkan informasi dari kelas induk.
Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
Versi penghitung biaya dengan parameter epoch tidak di-override.
4. Kelas turunan ketiga (model klasifikasi deep learning)
Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.
Constructor menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan constructor kelas di atasnya.
Override method: tampilkan informasi dari kelas di atasnya.
Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

Jawaban :

Model.java

```java
package unguided.unguided1;


public class Model {

    private String namaModel;

    private int jumlahDataLatih;


    public Model(String namaModel, int jumlahDataLatih) {

        this.namaModel = namaModel;

        this.jumlahDataLatih = jumlahDataLatih;

    }
```

```
    public String getNamaModel() {
        return this.namaModel;
    }

    public int getJumlahDataLatih() {
        return this.jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + this.namaModel + " | Data latih:
" + this.jumlahDataLatih);
    }

    public int hitungBiaya() {
        return this.jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return this.jumlahDataLatih * epoch;
    }
}
```

ModelDeepLearning.java

```
package unguided.unguided1;

public class ModelDeepLearning extends ModelKlasifikasi {
```

```
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public int getJumlahLayer() {
        return this.jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + this.jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahLayer * 200);
    }
}
```

ModelKlasifikasi.java

```
package unguided.unguided1;

public class ModelKlasifikasi extends Model {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public int getJumlahKelas() {
        return this.jumlahKelas;
```

```
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + this.jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * this.jumlahKelas;
    }
}
```

ModelRegresi.java

```
package unguided.unguided1;

public class ModelRegresi extends Model {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public int getJumlahFitur() {
        return this.jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + this.jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
```

```
        return super.hitungBiaya() + (this.jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return (getJumlahDataLatih() * epoch) / ukuranBatch;
    }
}
```

Main.java

```
package unguided.unguided1;

public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500,
4);
        ModelKlasifikasi klasifikasi = new ModelKlasifikasi("Decision
Tree", 800, 3);
        ModelDeepLearning deepLearning = new ModelDeepLearning("Neural
Network", 1000, 5, 3);

        Model[] daftarModel = { regresi, klasifikasi, deepLearning };

        int totalBiaya = 0;
        for (Model model : daftarModel) {
            model.tampilkanInfo();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): "
```

```
+ ((ModelRegresi) daftarModel[0]).hitungBiaya(10, 50));
    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

Program ini menerapkan konsep inheritance berjenjang dari kelas induk Model ke ModelRegresi dan ModelKlasifikasi yang kemudian diturunkan lagi ke ModelDeepLearning Konsep method overriding digunakan pada tampilkanInfo() dan hitungBiaya() untuk menyesuaikan formula biaya komputasi spesifik tiap jenis model, sedangkan method overloading digunakan untuk menghitung biaya berdasarkan variasi jumlah epoch dan ukuran batch. Pada kelas. Main seluruh objek diproses secara polimorfik dalam array bertipe kelas induk untuk menampilkan informasi tiap model, menghasilkan akumulasi total biaya sebesar 16440, serta memverifikasi biaya pelatihan dengan parameter tambahan secara tepat.

D. Kesimpulan

1. Inheritance adalah fondasi reusability dalam OOP.

Dengan extends, subclass otomatis mewarisi atribut dan metode superclass, sehingga kode tidak perlu ditulis ulang. Hubungannya bersifat "is-a" (contoh: Pria adalah Manusia), dan Java membatasi ini pada single inheritance untuk class.

2. Keyword super menjembatani komunikasi subclass ke superclass.

super memungkinkan subclass mengakses konstruktor, atribut, atau metode milik superclass terutama saat terjadi shadowing atribut atau saat subclass perlu memanggil versi asli metode yang telah di-override.

3. Overloading dan Overriding adalah dua konsep berbeda yang sering tertukar.

Overloading terjadi dalam satu kelas dengan nama metode sama tapi parameter berbeda (untuk fleksibilitas input).

Overriding terjadi antara superclass dan subclass dengan nama, parameter, dan tipe kembalian sama persis (untuk spesialisasi perilaku).

4. Inheritance membuka jalan menuju Polymorphism.

Karena subclass adalah bagian dari superclass, variabel bertipe superclass dapat menampung objek subclass (contoh: Manusia m = new Pria();). Ini memungkinkan satu perintah yang sama menghasilkan perilaku berbeda tergantung objek aslinya—inti dari polymorphism.